DATABASE TECHNOLOGIES LAB SHEET – 9

Course Code: MCA105P
Topic: MySQL – Student Record Management System

Q1. Install and configure MySQL and verify the MySQL connection using Command Line.

```
SHOW DATABASES;
```

Q2. Create a database named CollegeDB and a table named students. Insert at least 5 student records.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   gender VARCHAR(10),
   department VARCHAR(50),
   marks FLOAT
);
INSERT INTO students VALUES
(101, 'Aman', 21, 'Male', 'MCA', 85),
(102, 'Riya', 22, 'Female', 'BCA', 78),
(103, 'Rahul', 20, 'Male', 'MCA', 65),
(104, 'Priya', 23, 'Female', 'BBA', 90),
(105, 'Karan', 21, 'Male', 'BCA', 72);
SELECT * FROM students;
```

Q3. Create a student registration table containing Student ID, Student Name, Age, Gender, Department and Marks.

```
CREATE TABLE StudentRegistration (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   gender VARCHAR(10),
   department VARCHAR(50),
   marks FLOAT
);
```

Q4. Create a table with appropriate data types and constraints.

```
CREATE TABLE StudentDetails (
   student_id INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   age INT,
   gender VARCHAR(10),
   department VARCHAR(50),
   marks FLOAT
);
```

Q5. Connect to the CollegeDB database and access the students table.

```
USE CollegeDB;
SHOW TABLES;
DESC students;
```

Q6. Insert student information into the students table and verify the newly added record.

```
INSERT INTO students
VALUES (106, 'Neha', 22, 'Female', 'MCA', 88);
SELECT * FROM students;
```

Q7. Retrieve all student records and display Student ID, Name, Age, Gender, Department and Marks.

```
SELECT * FROM students;
SELECT student_id, name, age, gender, department, marks
FROM students;
```

Q8. Create a search facility to find student details using Student ID. Handle the situation where the student does not exist.

```
SELECT * FROM students
WHERE student_id = 101;
SELECT * FROM students
WHERE student_id = 999;
```

Q9. Update a student's Name, Age, Department and Marks using the UPDATE operation.

```
UPDATE students
SET
name = 'Aman Sharma',
age = 22,
department = 'MCA',
marks = 92
WHERE student_id = 101;
SELECT * FROM students
WHERE student_id = 101;
```

Q10. Delete a student record using Student ID and display the remaining records.

```
DELETE FROM students
WHERE student_id = 105;
SELECT * FROM students;
```

Q11. Perform filtering operations using MySQL queries.

```
-- (a) Students belonging to MCA department
SELECT * FROM students
WHERE department = 'MCA';

-- (b) Students having marks greater than 75
SELECT * FROM students
WHERE marks > 75;

-- (c) Students having marks between 50 and 80
SELECT * FROM students
WHERE marks BETWEEN 50 AND 80;

-- (d) Students older than 20
SELECT * FROM students
WHERE age > 20;
```

Q12. Create a complete Student Record Management System using MySQL Command Line with Add, View, Search, Update and Delete operations.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    gender VARCHAR(10),
    department VARCHAR(50),
    marks FLOAT
);

-- Add Student
INSERT INTO students
VALUES (107, 'Sakshi', 21, 'Female', 'MCA', 89);

-- View Students
SELECT * FROM students;

-- Search Student
SELECT * FROM students
WHERE student_id = 107;

-- Update Student
UPDATE students
SET marks = 95
WHERE student_id = 107;

-- Delete Student
DELETE FROM students
WHERE student_id = 107;

-- Verify Records
SELECT * FROM students;
```